SIMATS ENGINEERING

Department of Information Technology

CSA0405 – Operating Systems

ASSIGNMENT 1 – CO1

Evaluate and Justify the Operating-System Design of a Smart Hospital

CO1: Apply the concepts of operating system types, structures, and system calls to demonstrate their functions in various computing environments

Student Name	Mohammed Falah Miya	**Register Number**	1911260265
Department / Section		**Academic Year / Semester**	
Faculty Name		**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	05/11/2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Requirements of Each Environment	4		
2	Types of Operating Systems	3		
3	Operating-System Structures	4		
4	Operating-System Services	4		
5	System Calls	4		
6	Cost of System Calls (Complete the Code)	4		

Task	Task Title	Max	Marks Awarded	Remarks
7	Process Control System Calls (Complete the Code)	4		
8	CPU States and Protection	3		
9	Memory Hierarchy and I/O Systems	4		
10	Measurement and Comparison	4		
11	Final Recommendation	4		
12	Day-4 Design Challenge (A, B, C, D)	8		
	TOTAL	50		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence expected	Status	Note
1	Ranked requirement table for every environment with justification; conflicts; patient-safety critical requirements; historical OS lessons; alternative single-OS evaluation (worst-case alarm, memory, administration)		

2	Justified choice of OS type for each environment with rejected alternatives; SMP vs AMP; clustered fail-over; alternative single multiprocessor evaluation		
3	Comparison of OS structures against hospital requirements; defended choice for monitors and database server		
4	Service table for each environment with justified inclusions and exclusions		
5	Step-by-step explanation of a system call with strace evidence from a real program		
6	Measured system-call counts and times for different buffer sizes; completed code; justified recommendation		
7	Working program (fork/exec/wait/exit); process diagrams; zombie/orphan explanation; fflush rationale; alternative evaluation with timings		
8	Reasoned explanation of CPU modes, privileged instructions, interrupts/traps/timer and memory protection applied to monitors; polling vs interrupts evaluation		
9	Justified data-placement plan and I/O method (programmed / interrupt-driven / DMA) using typical access times; caching effect; interrupt-per-word vs DMA for 500 MB image		

MEASUREMENT & IMPLEMENTATION EVIDENCE CHAIN (Tasks 5–7)

Evidence item	Status	Observation
Source code (Task 6 & 7)		

Test inputs / datasets		
Expected behaviour		
Actual output / screen results		
Intermediate algorithmic / measurement evidence		
Boundary / failure tests		
Interpretation linked to theory		
Code, data and report consistent		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All tasks (1–9) plus Design Challenge answered	☐ Yes ☐ Partly ☐ No
Ranked requirement tables and justifications present (Task 1)	☐ Yes ☐ Partly ☐ No
OS-type choice table with rejected alternatives (Task 2)	☐ Yes ☐ Partly ☐ No
Structure comparison and defended choices for monitors + database (Task 3)	☐ Yes ☐ Partly ☐ No
Service inclusion/exclusion table (Task 4)	☐Yes ☐ Partly ☐ No
strace / system-call walk-through present (Task 5)	☐ Yes ☐ Partly ☐ No
Source code for every program that produced reported data (Tasks 6–7)	☐ Yes ☐Partly ☐ No
Tables and graphs / process diagrams for experimental results	☐Yes ☐ Partly ☐ No
Figures / screen outputs included in the submission (not only referenced)	☐ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☐ Yes ☐ Partly ☐ No
Work consistent throughout	☐Yes ☐ Partly ☐ No
No unfilled template placeholders	☐Yes ☐ Partly ☐ No

Design Challenge (remote + cloud) impact / evaluation / adaptation / defence complete	☐ Yes ☐ Partly ☐No
Viva readiness (able to run the programs and explain measurements)	☐Yes ☐ No

Operating Systems (CSA0405)

ASSIGNMENT 1

CO1: Apply the concepts of operating system types, structures and system calls

Evaluate and Justify the Operating-System Design of a Smart Hospital

Detail	Entry
Student name	Mohammed Falah Miya
Register number	1911260265
Course / Assessment	CSA0405 Operating Systems, Assignment 1 (CO1), 50 marks, individual
Bloom's level	K5 – Evaluate
Submission date	05/11/2026
GitHub repository	https://github.com/mdfalahmiya-lang/OS/tree/main
Compiler and version	gcc 15.2.0 (Ubuntu 15.2.0-16ubuntu1)
Linux distribution	Ubuntu 26.04.1 LTS (WSL2 on Windows)
Compile command	gcc -std=c11 -Wall -Wextra -pthread

Mark Distribution and Contents

Task	Bloom's level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		42
Task 12 – Day-4 Design Challenge	K5 – Evaluate	8
TOTAL		50

All source code, outputs and screenshots are in the GitHub repository named above (one folder per task, progressive commits). Programs were compiled and executed on Ubuntu 26.04.1 LTS (WSL2) with gcc 15.2.0.

Task 1 – Evaluate the Requirements of Each Computing Environment (4 Marks)

The hospital contains six different computing environments. Each is ranked below on five requirements (1 = most important). The ranking follows one rule: a requirement that can directly harm a patient outranks one that only costs time or money.

1.1 Ranked requirement table

Environment	Ranking (1 → 5)	Justification
Bedside monitor	1 Response time (hard 10 ms) 2 Reliability 3 Security/integrity 4 Battery/power 5 Throughput	A late alarm is a patient-safety failure, so determinism outranks everything. The monitor must also run for days without a restart (2). A tampered or corrupted reading is as harmful as a late one (3). It is mains-powered with a backup battery, so power matters less (4). It handles only a few hundred samples per second (5).
Nurse tablet	1 Battery 2 Security 3 Response time (soft) 4 Reliability 5 Throughput	A tablet that dies mid-shift is useless, so battery comes first. It is carried around and can be lost, so encryption and remote wipe rank second. Touch response of about 100 ms is only a convenience (3). A crash costs a reboot because data lives on the server (4).
Doctor workstation	1 Security 2 Response time (soft) 3 Reliability 4 Throughput 5 Battery	Many doctors share the machines, so login, per-user permissions and audit trails are the biggest concern. Interactive response is next. The machine is on mains power, so battery is irrelevant.
Database server	1 Reliability/availability 2 Security and integrity 3 Throughput 4 Response time 5 Battery	Records must never become unavailable (“must never stop”). Wrong or leaked records cause harm (2). The server must sustain transactions from 700+ clients (3). Tens of milliseconds per query is acceptable (4). A UPS covers power.
Imaging server	1 Throughput 2 Reliability 3 Security 4 Response time 5 Battery	MRI processing is a CPU-heavy batch workload, so total work per hour is what matters. A failed job can be restarted (2). Scans are patient data (3). Minutes of latency are acceptable (4).
Partner-hospital link	1 Security 2 Reliability 3 Throughput 4 Response time 5 Battery	Records cross an organisational boundary, so authentication and encryption come first. Messages must be queued and retried if the link drops (2). Bandwidth is moderate (3). Seconds of delay are acceptable (4).

1.2 Conflicting requirements

Conflict	Evaluation
Fast response vs many users on one machine	Time-sharing gives each of n users a slice, so worst-case wait grows with n. This is acceptable for workstations but not for a monitor, which therefore serves no users and runs only a few fixed tasks.
Throughput vs response time	Large time slices and batching raise throughput but delay interactive jobs. The imaging server favours throughput; the monitors favour response time.

Conflict	Evaluation
Security vs speed and battery	Encryption, authentication and auditing cost CPU and energy. On tablets this directly shortens battery life, so hardware-accelerated encryption is preferred.
Reliability vs updates	Patching normally needs a reboot. The database must never stop, so it needs a cluster with rolling updates.
Rich features vs small size	A full OS with every service is large and unpredictable. A monitor needs a small, fully analysable OS.

1.3 Requirements critical to patient safety

- **Alarm deadline of 10 ms on the monitor:** a missed alarm can cost a life. This is the only hard real-time requirement in the hospital.
- **Availability of the patient database:** doctors need allergy, drug and history data at all hours.
- **Integrity of records and readings:** a wrong dosage or a mixed-up patient record is dangerous.
- **Confidentiality of records:** a legal and ethical requirement; it is safety-relevant because patients avoid care if they distrust the system.

Battery life of tablets and the throughput of the imaging server are important for efficiency but are not directly life-critical.

1.4 Lessons from the history of operating systems

Generation	Key idea	Where it applies in the hospital
Batch	Jobs are queued and run one after another without user interaction, to keep the CPU busy.	Imaging server: MRI reconstruction jobs are queued and run for throughput, not interactivity.
Time-sharing	The CPU is switched quickly between users; protection between users becomes essential.	Doctor workstations and the database server, with many users and per-user protection.
Personal	One user per machine; responsiveness over utilisation.	Workstations: single-user comfort, but with the protection lessons of time-sharing.
Mobile	Power-aware, touch-driven, sandboxed apps, always connected.	Nurse tablets: aggressive sleep, app sandboxes, remote wipe.
Real-time / embedded	Correctness depends on meeting deadlines.	Bedside monitors with a guaranteed 10 ms alarm path.
Distributed	Machines cooperate over a network and fail independently.	Partner link and group of 3 hospitals.

1.5 Alternative concept – one general-purpose OS (GPOS) on every device

Solution if one GPOS is used everywhere: every monitor, tablet, workstation and server runs the same time-sharing OS (for example a desktop Linux or Windows image). The same image is easy to manage, but its scheduler is built for fairness, not for deadlines.

Criterion	GPOS everywhere	Separate OS per environment (my design)
Worst-case alarm response	Unbounded. With n runnable tasks and quantum q, a task can wait up to about (n − 1) × q. For n = 20 and q = 10 ms that is 190 ms, which is 19× over budget. Page faults, disk I/O and kernel locks add more. Complexity O(n).	Bounded. A priority-preemptive RTOS runs the alarm task as soon as the interrupt arrives: latency = interrupt latency + context switch + alarm task time, typically well below 1 ms and independent of the number of other tasks, O(1).

Criterion	GPOS everywhere	Separate OS per environment (my design)
Memory per device	About 1–4 GB RAM and 10–30 GB storage per device, including services the monitor never uses. Total grows as O(D × M_GPOS) with D devices.	Monitor: about 0.1–16 MB RAM (RTOS). Tablets: mobile OS. Servers: full OS only where needed.
Administration effort	One image, one skill set and one patch process. But a single flaw or bad update hits all 700+ devices at once (common-mode failure) and needs frequent patching and reboots.	Several OS families and more skills required. Faults are contained, and the RTOS on monitors changes rarely.

Conclusion: a single GPOS cannot guarantee the 10 ms alarm, wastes memory on every monitor, and puts all devices at risk from one fault, so the clinical staff's concern is justified. Its only advantage, simpler administration, does not outweigh a missed alarm.

Task 2 – Judge the Type of Operating System for Each Environment (3 Marks)

2.1 Types of operating system compared

Type	Strengths	Weaknesses for this hospital
Mainframe	Very high reliability, huge I/O, batch and transaction processing.	Very expensive and unnecessary for 500 beds. Not suitable for bedside or handheld use.
Desktop	Responsive GUI, wide software support.	No timing guarantee; weak fault tolerance; not for servers.
Multiprocessor	Higher throughput and some fault tolerance with several CPUs.	Needs a careful design (SMP vs AMP); still a single machine.
Distributed	Machines cooperate across a network; resource sharing.	Network delay and partial failure; harder to secure and to keep consistent.
Clustered	Fail-over and load sharing among several nodes.	Needs shared storage or replication; costlier to set up.
Hard real-time	Deadlines are guaranteed.	Small feature set; not for general use.
Soft real-time	Critical tasks get priority but no absolute guarantee.	Not safe for life-critical alarms.
Handheld	Low power, touch UI, small memory.	Limited CPU and storage; not for servers.

2.2 Chosen type for each environment, with rejected alternatives

Environment	Chosen type	Reasons and rejected alternatives
Bedside monitors	Hard real-time OS (RTOS)	The 10 ms alarm deadline needs guaranteed latency. Rejected: desktop OS and soft real-time (no absolute guarantee) and any GPOS.
Nurse tablets	Handheld/mobile OS	Designed for battery life, touch and sandboxed apps. Rejected: desktop OS (battery and size).
Doctor workstations	Desktop (time-sharing, multi-user) OS	Interactive use, rich applications, per-user login. Rejected: RTOS (poor application support).
Database server	Clustered system with fail-over	Continuous availability: if one node fails another takes over within seconds. Rejected: single multiprocessor (single point of failure) and mainframe (cost).
Imaging server	Multiprocessor (SMP) server OS	MRI processing is parallel and all cores are equal. Rejected: desktop OS (limited CPU scaling) and distributed (not needed inside one box).
Partner link	Distributed system (client–server/peer-to-peer messaging)	Independent hospitals exchange records over a network, needing authentication and encryption. Rejected: a shared single machine (organisational boundary).

2.3 Symmetric versus asymmetric multiprocessing for the imaging server

Aspect	Symmetric (SMP)	Asymmetric (AMP)
Design	Every processor runs the OS and any task.	A master processor runs the OS and assigns work to slave processors.
Load balance	Good: any CPU takes the next MRI slice.	Can bottleneck at the master.
Fault tolerance	If a CPU fails, the others continue.	If the master fails the system stops.

Aspect	Symmetric (SMP)	Asymmetric (AMP)
Complexity	Needs locking of kernel data.	Simpler to build.

Decision: SMP. MRI reconstruction splits into many independent slices, so all cores can work equally and no single master limits throughput.

2.4 Clustered database server with fail-over

Two or more nodes hold a replica of the patient database. A heartbeat message is exchanged every second or so. When the active node stops responding, the standby node takes over the service (virtual IP address and storage), so clients reconnect within seconds. Rolling updates are done node by node, so updates never stop the service. This answers the clinical concern that the database may stop during updates.

2.5 Alternative concept – one large multiprocessor instead of a cluster

Solution: run the database on a single large SMP machine with many cores and a lot of memory, with no second machine.

Criterion	Single large multiprocessor	Cluster with fail-over
Availability	Single point of failure. If one machine is 99.9% available, downtime is 0.1% × 8,760 h ≈ 8.8 h per year, and an update means an outage.	Two independent nodes: availability = 1 − (1 − 0.999)² = 99.9999%, about 32 seconds per year, plus the fail-over time of a few seconds per fault.
Cost	Cost grows faster than linearly with size (large memory and many sockets are expensive); the machine is a premium part.	Several cheaper commodity machines, but twice the hardware plus cluster software and licences.
Throughput vs processors (time)	Amdahl's law: $S(p) = 1 / ((1 - f) + f/p)$. With $f = 0.95$, $S(8) \approx 5.9$, $S(32) \approx 12.5$ and the limit is 20. Lock contention in one OS adds to the loss.	Read-heavy queries can be spread over nodes, so scale-out is close to linear in the number of nodes. Replication and coordination add overhead per write.
Memory (space)	One large shared memory, simple to use and no duplication.	Each node needs its own memory; data is stored in at least 2 copies, so storage is O(k × data) for k replicas.

Conclusion: the cluster is preferred, because the key requirement is availability, and no single machine can offer fail-over within seconds.

Task 3 – Evaluate Operating-System Structures (4 Marks)

3.1 Structures compared

Structure	Reliability	Performance	Ease of updating
Simple (monolithic)	Low. A fault in any part of the kernel can crash everything.	High. Direct function calls, no message passing.	Hard. The kernel is rebuilt as a whole.
Layered	Good. Each layer uses only the layer below, so it is easy to verify.	Lower. A request passes through several layers.	Moderate. Layers are replaced separately.
Microkernel	Very high. Only a tiny kernel runs in kernel mode; drivers and file systems are user processes that can crash and restart.	Lower. Services communicate by message passing.	Easy. Servers are replaced without a reboot.
Modular (loadable kernel modules)	Medium. Modules run in kernel mode, so a bad module can still crash the system.	High. Same as monolithic after loading.	Easy. Modules are loaded and unloaded at run time.
Hybrid	Medium to high.	High. Performance-critical parts stay in the kernel.	Moderate to easy.

3.2 Structure of real systems

System	Structure	Notes
Linux	Monolithic kernel with loadable modules	File systems, drivers and network stacks all run in kernel mode, and modules can be added at run time.
Windows	Hybrid	A microkernel-like core with executive services, drivers and the window manager running in kernel mode for speed.
Android	Layered (modified Linux kernel + libraries + runtime + framework + apps)	Apps run in separate sandboxed processes on top of a Linux kernel.
FreeRTOS	Minimal kernel library	Scheduler, queues and semaphores linked with the application into a single image, a few KB in size.
QNX	Microkernel	A small kernel handling scheduling and message passing; drivers and services are user-space processes.

3.3 Choice for two environments

- **Patient monitors – microkernel RTOS (QNX-style) or a minimal RTOS such as FreeRTOS:** a faulty sensor or display driver must not kill the alarm task. In a microkernel the driver is a restartable user process, and the kernel is small enough to be analysed for worst-case latency.
- **Database server – modular monolithic kernel (Linux with loadable modules):** the database needs the best file-system and network performance, and a mature, well-tested driver base. Modules allow new storage or network drivers to be loaded without a reboot, and the cluster takes care of a node crash.

3.4 Alternative concept – monolithic kernel on the bedside monitors

Solution: the monitor firmware would run a monolithic kernel in which sensor, display and network drivers are linked into one kernel address space, and the alarm task calls them through ordinary system calls.

Criterion	Monolithic kernel	Microkernel
Time cost of a service request	One system call: one trap and one mode switch. Measured on my Linux test machine: about 0.1–0.16 µs per trivial system call. O(1) per request.	Message passing: client → kernel → server → kernel → client, with at least two messages and extra context switches. Typically 2–10× slower, about 1–5 µs. Still O(1) per request.
Kernel memory	Large (a minimal Linux kernel is still about 1–2 MB, a full one tens of MB), because drivers and file systems are included.	Very small kernel (tens of KB), with services as separate processes whose total may be larger.
Effect of a faulty driver	The driver runs in kernel mode and can overwrite kernel memory, so the whole monitor crashes and every alarm is lost until reboot.	The driver crashes alone. A supervisor restarts it in milliseconds and the alarm task keeps running.

Conclusion: the microkernel's extra 1–5 µs per request is only about 0.05% of the 10 ms budget, whereas a driver fault in a monolithic kernel can silence all alarms. For safety-critical monitors, fault isolation is worth more than raw speed.

Task 4 – Evaluate the Services an Operating System Must Provide (4 Marks)

4.1 Service table (✔ = provided, ✘ = removed, ◖ = reduced)

Service	Monitor	Tablet	Work-station	Database server	Imaging server	Partner link
User interface	◖ minimal display	✔ touch GUI	✔ GUI	◖ CLI/admin only	◖ CLI	✘
Program execution	◖ fixed tasks	✔	✔	✔	✔	✔
I/O operations	✔ sensor, display	✔	✔	✔ storage, network	✔ MRI, storage	✔ network
File system	◖ small log only	✔ encrypted	✔	✔ journalled	✔ large files	◖ queue
Communication	✔ to nurse station	✔ Wi-Fi	✔	✔	✔	✔ secure
Error detection	✔ watchdog, self-test	✔	✔	✔ ECC, replication	✔	✔
Resource allocation	◖ fixed priorities	✔ power-aware	✔	✔	✔ CPU/memory	✔
Accounting	✘	◖	◖	✔ audit	◖ job logs	✔ audit
Protection and security	◖ integrity, signed firmware	✔ encryption, wipe	✔ users, ACLs	✔ strong	✔	✔ TLS, auth

4.2 Services removed from the bedside monitor

A bedside monitor can drop the multi-user login and graphical desktop, general file system (a small circular log remains), dynamic program loading, accounting, printing and most network protocols. This makes the OS image smaller, reduces the number of tasks competing for the CPU, and removes code that could fail or be attacked, which makes it faster and easier to prove that the alarm deadline is met.

4.3 Essential services for the database server

- **File system** with journalling for crash-safe writes, and **I/O** with caching and scheduling for throughput.
- **Protection and security:** user authentication, access control lists, encryption of data at rest and in transit.
- **Error detection and recovery:** ECC memory, RAID or replication, and fail-over to the standby node.
- **Resource allocation** for many concurrent connections, **accounting** and audit logs (who read which record), and **communication** with clients and the partner link.

4.4 How one patient's records are protected from another user's programs

- **Authentication:** each user logs in and is given a user ID.
- **File protection:** each record file or table has an owner and permissions or an access control list, and the OS checks them on every open().
- **Memory protection:** the MMU gives every process its own address space with base/limit or page-table checks, so one program cannot read another's memory.
- **Mode separation:** programs run in user mode and must use system calls, so the kernel can verify each request.
- **Encryption and audit:** records are encrypted on disk and every access is logged.

4.5 Alternative concept – full desktop OS with every service on the tablets

Solution: each nurse tablet runs a desktop OS with all services (multi-user logon, indexing, printing, background updates, full GUI).

Criterion	Full desktop OS on tablet	Reduced handheld OS
Memory (space)	About 4–8 GB RAM in use for the OS and services; more total RAM must be fitted, adding weight and cost.	About 1–2 GB, with unused services removed and background apps frozen.
Storage (space)	About 30–60 GB for the OS image.	About 5–10 GB.
Start-up time	Longer (typically 30–60 s cold boot), because many services start.	Shorter (typically 10–20 s), and normally the tablet just sleeps and wakes.
Battery life (time)	Background services and a power-hungry CPU keep the device awake; battery typically falls to about half of what a handheld OS achieves, so it may not last a shift.	Aggressive sleep and per-app power management give a full shift of 8–12 h.

(The figures are typical engineering estimates, not measurements from my machine.) **Conclusion:** the handheld OS meets the clinical staff's battery concern, and the extra services of a desktop OS would not be used by nurses.

Task 5 – Examine System Calls (4 Marks)

5.1 Categories of system calls

Category	Purpose	Examples
Process control	Create, run, end and wait for processes; allocate memory.	fork, execve, wait, exit, brk/mmap
File management	Create, open, read, write, close and delete files.	open, read, write, close, unlink
Device management	Request, use and release devices.	ioctl, read/write on device files
Information maintenance	Get or set system and process data.	getpid, time, gettimeofday, uname
Communication	Pipes, sockets, shared memory, message passing.	pipe, socket, send, recv, shmget
Protection	Control access to resources.	chmod, chown, umask, setuid

5.2 What happens when a program calls read(fd, buf, n)

Step	Action
1	Library call: the program calls the C library wrapper read(). Like any function call it pushes a return address, but this is not yet the kernel.
2	Parameters set up: the wrapper puts the system-call number (0 for read on x86-64 Linux) and the arguments (fd, buf, n) into registers.
3	Trap: the wrapper executes the syscall instruction. The CPU raises a software interrupt (trap) and jumps to the kernel's fixed entry point.
4	Mode switch: the hardware switches from user mode to kernel mode (mode bit = 0) and the kernel stack is used.
5	Kernel work: the kernel uses the number to index the system-call table, validates fd and the buffer address, finds the file in its open-file table, checks permissions, and either copies data from the page cache or starts a disk read and blocks the process until the disk interrupt arrives. The data is copied into the user buffer.
6	Return: the kernel stores the return value (bytes read, 0 at end of file, or −1 with errno) in a register, restores user state, and executes the return instruction (sysret). The mode bit is set back to user mode.
7	Resume: the library wrapper returns the value to the program, which continues after the call.

5.3 How parameters are passed to the kernel

Method	How it works	Used when
Registers	Number and arguments are placed in CPU registers. This is the fastest method.	Few parameters (Linux x86-64 passes up to 6 in registers).
Block (table) in memory	Parameters are stored in a memory block and the block's address is passed in a register.	More parameters than registers, or large structures (Linux and Solaris use this for some calls).
Stack	The program pushes the parameters and the kernel pops them.	Some older systems; the kernel must check that the user stack is valid.

5.4 Evidence: strace on a real program

The program Task05/task05_demo.c writes a small patient record with the raw system calls, reads it back with read(), and then reads it again with fopen/fread. It was compiled with gcc 15.2.0 and run on Ubuntu 26.04.1 LTS (WSL2) (output as obtained):

```
raw read() got 27 bytes: patient=1042;hr=72;spo2=98
fread() got 27 bytes
pid=4655
```

Commands used to obtain the system-call trace:

```
gcc -std=c11 -Wall -Wextra -pthread task05_demo.c -o task05_demo
strace -o trace.txt ./task05_demo
grep -E "openat|read|write|close" trace.txt
strace -c ./task05_demo
```

The relevant lines of trace.txt have this form (the file descriptor is 3 because 0, 1 and 2 are stdin, stdout and stderr). My captured trace is shown first, then the annotated form of the same calls:

My trace (Task05/trace_demo.txt, from strace -o trace_full.txt ./task05_demo):

```
  openat(AT_FDCWD, "record.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
  write(3, "patient=1042;hr=72;spo2=98\n", 27) = 27
  close(3)                                = 0
  openat(AT_FDCWD, "record.txt", O_RDONLY) = 3
  read(3, "patient=1042;hr=72;spo2=98\n", 63) = 27
  close(3)                                = 0
  fstat(1, {st_mode=S_IFCHR|0666, st_rdev=makedev(0x1, 0x3), ...}) = 0
  ioctl(1, TCGETS2, 0x7ffe69b1df40)       = -1 ENOTTY (Inappropriate ioctl for device)
  brk(NULL)                               = 0x5a5d750f6000
  brk(0x5a5d75118000)                     = 0x5a5d75118000
  openat(AT_FDCWD, "record.txt", O_RDONLY) = 3
  fstat(3, {st_mode=S_IFREG|0644, st_size=27, ...}) = 0
  read(3, "patient=1042;hr=72;spo2=98\n", 4096) = 27
  read(3, "", 4096)                       = 0
  close(3)                                = 0
  getpid()                                = 4666
  unlink("record.txt")                    = 0
  write(1, "raw read() got 27 bytes: patient"..., 82) = 82
  exit_group(0)                           = ?
  +++ exited with 0 +++
```

```
openat(AT_FDCWD, "record.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "patient=1042;hr=72;spo2=98\n", 27) = 27
close(3)                                 = 0
openat(AT_FDCWD, "record.txt", O_RDONLY) = 3
read(3, "patient=1042;hr=72;spo2=98\n", 63) = 27
close(3)                                 = 0
...
openat(AT_FDCWD, "record.txt", O_RDONLY) = 3
read(3, "patient=1042;hr=72;spo2=98\n", 4096) = 27   <- fread(): 4096-byte buffer
```

Reading the trace: each line is one trap into the kernel. The first read() asks for 63 bytes because the program passed sizeof buf − 1. The library's fread() asks for 4096 bytes, the size of its internal buffer, even though the program asked for 63. This is the buffering described in the alternative-concept analysis below.

5.5 Alternative concept – C library functions (fopen, fread, fwrite) instead of direct system calls

Solution: the program calls fopen/fread/fwrite. The library keeps a user-space buffer (typically 4096 bytes) and calls read() or write() only when the buffer is empty or full.

```
/* direct system calls: n/b read() calls */
while ((k = read(fd, buf, b)) > 0) { process(buf, k); }

/* library: about n/4096 read() calls, however small the fread() requests */
while (fread(&ch, 1, 1, fp) == 1) { process(&ch, 1); }
```

Criterion	Direct read()/write() with buffer b	fopen/fread/fwrite
System calls for n bytes	n / b (for b = 1 this is n calls). O(n/b).	About n / 4096, independent of how many bytes each fread() requests. O(n/4096).

Criterion	Direct read()/write() with buffer b	fopen/fread/fwrite
Extra buffer space	Only the program's own buffer of b bytes.	An extra library buffer of about 4 KB (heap) per open file, so O(b) per file, plus one copy from library buffer to program buffer.
Measured (4 MiB file, 1 byte per request)	4,194,305 read() calls; 3.145 s wall time (2.521 s kernel time).	1-byte fread/fwrite: 0.125 s wall time, which is about 25× faster, because only about 1,025 read() calls reach the kernel.

Conclusion: the library is the better choice when the program naturally works in small pieces. Direct system calls are better when the program already uses a large buffer, or needs exact control of when data reaches the device (for example flushing a log entry).

Task 6 – Evaluate the Cost of System Calls (4 Marks)

6.1 Why a system call costs much more than a function call

- A function call is a jump plus a return address push. A system call needs a trap, a mode switch, saving and restoring registers, validation of the arguments, and often a flush of CPU pipelines and caches.
- Security mitigations on modern CPUs add further cost on each kernel entry.
- Blocking calls such as read() can also cause a context switch while the process waits for the disk.

Measurement (Task05/syscall_cost.c, 5,000,000 iterations):

Operation	Time per call	Ratio
Ordinary function call (not inlined)	about 1.4 ns	1×
System call (getppid)	about 150 ns	about 107×

A system call that only returns a number is already about 40 times as expensive as a function call. A call that copies data or touches the disk costs far more.

6.2 Complete the Code – copy program with read() and write()

The only change is the code that replaces TODO 1; the rest of the program is unchanged. File: Task06/task06_copy.c.

```
/* TODO 1 - completed */
while (1) {
    n = read(in, buf, bufsize);
    calls++;                        /* count every read() call           */
    if (n <= 0) break;              /* 0 = end of file, -1 = error       */
    if (write(out, buf, n) != n) { /* write exactly the n bytes read    */
        close(in); close(out);
        return -1;
    }
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra -pthread task06_copy.c -o task06
./task06
```

Output obtained (matches the expected output):

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Why these numbers: the file has 10,000 bytes. With a 1-byte buffer there are 10,000 calls that return data plus one final call that returns 0, so 10,001. With 512 bytes: ⌈10,000 / 512⌉ = 20 calls with data plus the final call = 21. With 4096 bytes: ⌈10,000 / 4096⌉ = 3 plus the final call = 4. In general the number of calls is ⌈n / b⌉ + 1.

Comparison with strace -c: run strace -c ./task06 and read the “calls” column for read. It should be the program’s total of 10,001 + 21 + 4 = 10,026, plus one or two extra reads made by the dynamic loader when the program starts. Write-calls should be 10,000 + 20 + 3 = 10,023 for the copies, plus the few writes made by fputc/printf. My strace -c output is shown below: the read() count is 10027.

strace -c ./task06 (Task06/strace_c.txt):

```
  % time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 50.54    0.195486          19     10027           write
 48.89    0.189077          18     10027           read
  0.17    0.000662          73         9           openat
  0.10    0.000375          41         9           close
  0.07    0.000271         135         2           unlink
  0.06    0.000215          26         8           mmap
  0.05    0.000202         101         2           pread64
  0.04    0.000144          36         4           fstat
  0.02    0.000096          32         3           mprotect
```

```
  0.02    0.000059          19         3           brk
  0.01    0.000050          50         1         1 access
  0.01    0.000023          23         1           munmap
  0.01    0.000021          21         1           rseq
  0.00    0.000019          19         1         1 ioctl
  0.00    0.000019          19         1           getrandom
  0.00    0.000018          18         1           arch_prctl
  0.00    0.000015          15         1           prlimit64
  0.00    0.000014          14         1           set_tid_address
  0.00    0.000013          13         1           set_robust_list
  0.00    0.000000           0         1           execve
------ ----------- ----------- --------- --------- ----------------
100.00    0.386779          19     20104         2 total
```

6.3 Effect of buffer size and library buffering

Measured copy of a 4 MiB file with read()/write() (Task10/task10_bench.c; Ubuntu 26.04.1 LTS (WSL2), gcc 15.2.0, 20 vCPUs; times in seconds):

Buffer (bytes)	read() calls	Wall time	User time	Kernel time
1	4,194,305	3.145	0.592	2.521
16	262,145	0.189	0.035	0.154
64	65,537	0.097	0.012	0.047
512	8,193	0.016	0.000	0.010
4,096	1,025	0.009	0.000	0.004
32,768	129	0.006	0.000	0.003
65,536	65	0.005	0.000	0.003
1,048,576	5	0.007	0.000	0.006

Going from a 1-byte to a 4096-byte buffer cuts the system calls by about 4,096× and the time by about 350×. Beyond 4–64 KB the curve is flat, because per-call overhead is now small compared with the time to copy the data. Library buffering (fread/fwrite) achieves a similar effect automatically: 1-byte fread/fwrite on the same file took 0.125 s versus 3.145 s for 1-byte read/write.

Recommended buffer size for the hospital backup program: 64 KB. It sits at the knee of the measured curve (0.005 s, only 65 read calls), is a multiple of the 4 KB page and file-system block size, and uses little memory. Larger buffers (1 MiB) bring no measurable gain here and waste memory when several backups run in parallel.

6.4 Alternative concept – memory-mapped files (mmap) for the backup

Solution: the backup program maps the source file into its address space with mmap(), maps the destination, and copies with memcpy(). The kernel brings pages in on demand as page faults.

Criterion	read()/write() with buffer b	mmap
Number of system calls	About n / b (O(n/b)).	A few: open, fstat, mmap, munmap, close (O(1)); the page faults are handled by the kernel and are not system calls.
Copying kernel ↔ user	Data is copied from the page cache to the user buffer, and back for write().	No copy into a user buffer for reads, because the process reads the page-cache pages directly; memcpy to the destination mapping is the only copy.
Memory for very large files	Constant: just the b-byte buffer, whatever n is.	Needs n bytes of virtual address space, and pages are loaded as touched; resident memory is managed by the kernel and can be evicted. Fine on a 64-bit system, but an I/O error appears as a signal (SIGBUS) rather than a return value.

Criterion	read()/write() with buffer b	mmap
Measured (4 MiB)	0.005–0.009 s with a 4–64 KB buffer.	0.060 s. On a small file the page-fault cost outweighs the saved copy.

Conclusion: mmap reduces the number of system calls but is not automatically faster. For a sequential backup, read()/write() with a 64 KB buffer is simpler and just as fast.

Task 7 – Evaluate Process Control System Calls (4 Marks)

7.1 What fork(), exec(), wait() and exit() do

Call	Function
fork()	Creates a child process that is a copy of the parent (same code, data copy-on-write, same open files). Returns the child's PID in the parent, 0 in the child, and −1 on error.
exec()	Replaces the calling process's program with a new program. The PID stays the same; code, data and stack are replaced. It returns only on failure.
wait()/waitpid()	Blocks the parent until a child ends, and returns the child's exit status (macros such as WEXITSTATUS extract it). It also removes the child's process-table entry.
exit()	Ends the calling process, flushes stdio buffers, closes files and passes the exit status to the parent. (_exit() ends immediately without flushing stdio buffers.)

7.2 How a shell runs a command and gets its exit status

```
shell (parent, pid 100)
   |  fork()
   +-------------------------> child (pid 101, copy of shell)
   |                               |  execlp("expr","expr","6","*","7")
   |  waitpid(101, &status, 0)     |  -> program replaced by expr
   |  (parent blocks)              |  expr prints 42
   |                               |  exit(0)
   |<------ status = 0 -----------+
   |  prints "job 1 exit status 0" (WEXITSTATUS(status))
   v  next command
```

The shell forks, the child execs the command, and the parent calls waitpid(). The 8 bits of exit status (0 = success) are extracted with WEXITSTATUS(status) and stored by the shell, for example in $?.

7.3 Zombie and orphan processes

- **Zombie:** a child that has terminated but whose parent has not yet called wait(). Only its process-table entry (PID, exit status) remains. The parent's wait() removes it. A parent that never waits leaks table entries.
- **Orphan:** a child whose parent has terminated first. The OS re-parents it to init/systemd (PID 1), which regularly calls wait() and therefore cleans up the child when it ends.

7.4 Why fork() followed by exec() rather than one "create process" call

- Between fork and exec, the child (still running the shell's code) can set up redirection, pipes, working directory, environment and signals. A single create-process call would need dozens of parameters for all of these.
- Two simple, orthogonal calls can be combined freely: fork alone (a server handling a client) or exec alone (replace the current program).
- Copy-on-write makes fork cheap, since pages are copied only when changed.

7.5 Complete the Code – a shell that runs two jobs

File: Task07/task07_shell.c. The completed code for TODO 1:

```
pid_t pid;
int status;
pid = fork();
if (pid < 0) { perror("fork"); return 1; }
if (pid == 0) {
    execlp("expr", "expr", "6", "*", "7", (char *)NULL);
    exit(127);                      /* only reached if exec fails */
}
waitpid(pid, &status, 0);
printf("job 1 exit status %d\n", WEXITSTATUS(status));
```

The original program uses the variables pid and status after the TODO without declaring them, so their declarations belong inside TODO 1. Nothing outside the TODO was changed. Compile and run:

```
gcc -std=c11 -Wall -Wextra task07_shell.c -o task07
./task07
```

Output obtained (matches the expected output):

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Job 1 exits with status 0 because expr returns 0 when its result (42) is neither zero nor empty. Job 2 reports 3 because the child ran _exit(3).

7.6 Why stdout is flushed before each fork()

stdout is line-buffered on a terminal but fully buffered when redirected to a file or pipe. A fork() copies the process's memory, including any characters still waiting in the stdio buffer. If the buffer is not empty at the time of the fork, both parent and child own a copy and each will eventually write it, so output appears twice. fflush(stdout) empties the buffer first, so there is nothing to duplicate.

7.7 Experiment: remove the second fflush and use exit(3)

I removed the second fflush(stdout) and changed _exit(3) to exit(3) (file Task07/task07_dup.c) and redirected the output to a file (./task07_dup > out.txt). The file contained:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explanation: with output redirected to a file, stdout is fully buffered. After job 1 the buffer holds the text "job 1 exit status 0" and "shell: running job 2", which has not been flushed. The second fork() copies this buffer into the child. The child then calls exit(3), which (unlike _exit) flushes stdio buffers, so the child writes the copied text to the file, and the parent later writes its own copy too. Hence "shell: running job 2" (and the "job 1 exit status 0" line before it) appears twice. With _exit(3) the child would have skipped the flush and nothing would be duplicated.

7.8 Alternative concept – posix_spawn() or threads instead of fork()+exec()

Criterion	posix_spawn / threads	fork() + exec()
Time to start 1,000 commands	posix_spawn of /bin/true: 0.603 s (about 0.6 ms each). Threads: 0.081 s for 1,000 (about 81 µs each), but a thread cannot run a separate program. O(k) for k commands in each case.	fork()+exec of /bin/true: 0.679 s (about 0.7 ms each). fork() alone: 0.223 s. O(k).
Memory copied or shared	posix_spawn typically avoids copying the parent's page tables (uses vfork-like sharing until exec). Threads share all memory.	fork copies the page tables and marks pages copy-on-write; exec then discards them. Cost grows with parent size.
Isolation between commands	Threads share address space and file descriptors: one crash or bug can kill the whole shell. posix_spawn gives full process isolation.	Full process isolation, and the child can set up redirection before exec.

Conclusion: the time is dominated by loading the new program (about 0.6 ms), not by fork, so posix_spawn gives only a small gain (about 11%) for a shell. Threads are faster but give no isolation and cannot run another executable, so fork+exec remains the right design for a command shell.

Task 8 – Evaluate CPU States and Protection (3 Marks)

8.1 User mode, kernel mode and the mode bit

The CPU has a mode bit that selects user mode (1) or kernel mode (0). In kernel mode all instructions are allowed. In user mode, privileged instructions (I/O instructions, changing the memory-management registers, disabling interrupts, halting the CPU, changing the mode bit) cause a trap if attempted. User programs use system calls to ask the kernel to do these things. The hardware is therefore the root of trust: even a buggy or malicious program cannot access devices directly.

8.2 Interrupts, traps and the timer interrupt

Event	Cause	Example in the hospital
Interrupt	Asynchronous signal from hardware.	The sensor's analogue-to-digital converter signals a new reading; the network card receives a packet.
Trap (exception)	Synchronous, caused by the running program: error or system call.	Divide by zero, invalid memory access, or the syscall instruction.
Timer interrupt	A hardware timer fires at a fixed interval.	On workstations it lets the OS take the CPU back from a looping program. On monitors it drives the periodic sampling and the watchdog.

8.3 What if any program could disable interrupts?

Disabling interrupts is a privileged instruction for a reason. On the monitor, a faulty or malicious program that disables interrupts would stop the sensor interrupt, so abnormal readings would not be noticed and the alarm would never be raised. The timer interrupt would also stop, so the OS could not preempt the program, and the monitor would hang. On the workstations the same would let a user program monopolise the CPU. The mode bit prevents this: only the kernel may disable interrupts, and only for very short critical sections.

8.4 How the hardware stops one program reading another program's memory

- **Base and limit registers:** every address is checked to lie between base and base + limit; otherwise the hardware traps. Only the kernel can load them.
- **Paging with an MMU:** every process has its own page table, so its virtual addresses map only to its own physical frames; permission bits (read, write, execute) are checked on each access. An invalid access raises a page-fault trap and the OS ends the offending process.

A program therefore cannot read, or even name, another patient-record program's memory.

8.5 Alternative concept – polling instead of interrupts on the monitors

Solution: the monitor's main loop reads the sensor repeatedly (every T ms) and compares the value with the alarm threshold, instead of sleeping until the sensor raises an interrupt.

```
while (1) {
    v = read_sensor();          /* poll */
    if (v > LIMIT) raise_alarm();
    sleep_until_next_poll(T);
}
```

Criterion	Polling	Interrupt-driven
Worst-case delay before an abnormal reading is noticed	Up to the polling interval T (plus processing). To meet 10 ms, T must be well below 10 ms, for example 2 ms. Delay O(T).	Interrupt latency, typically a few microseconds to tens of microseconds, plus handler time. Independent of any polling interval.

Criterion	Polling	Interrupt-driven
CPU time wasted	Each poll costs CPU time even when nothing has happened. With T = 2 ms the CPU wakes 500 times per second, and a busy-wait loop would use 100% of the CPU.	No CPU is used while idle; the handler runs only when an event occurs.
Power	The CPU cannot stay in deep sleep, so the energy use is higher.	The CPU sleeps until the interrupt (wait-for-interrupt instruction), so the energy use is lower.

Conclusion: interrupts give a faster, bounded reaction and lower power. Polling is only acceptable where the device is always busy anyway, or the event rate is very high.

Task 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks)

9.1 Memory hierarchy with typical access times and sizes

Level	Typical access time	Typical size	Notes
Registers	< 1 ns	Hundreds of bytes	Fastest and smallest; managed by the compiler.
Cache (L1/L2/L3)	1 ns / 3–5 ns / 10–20 ns	32–64 KB / 256 KB–2 MB / 8–64 MB	Hardware-managed; volatile.
Main memory (DRAM)	60–100 ns	Several GB to TB	Volatile; holds running programs.
SSD / NVMe	20–150 µs	Hundreds of GB to several TB	Non-volatile; no moving parts.
Hard disk	5–10 ms	Several TB	Cheap and large; slow.
Network storage	0.5–10 ms and more	TB to PB	Shared, replicated, easy to back up; depends on the network.

Each step down is roughly 10× to 1,000× slower, larger and cheaper per byte, so frequently used data must be kept near the top.

9.2 Where data should be kept

Data	Placement and reason
MRI images	The scan being processed (hundreds of MB) sits in RAM of the imaging server for speed; recent scans stay on NVMe SSD for quick viewing; older scans are archived to large, replicated network storage or HDD, because they are big, rarely accessed and must be kept for years.
Patient records	The database keeps hot records in its RAM buffer cache; the database files live on SSD for fast random reads and writes with journalling; backups and replicas go to network storage. Not on a single HDD, which is slow and a single point of failure.
Monitor readings	Live samples are held in a small RAM/SRAM ring buffer on the monitor (access in ns, no disk delay). A short history is written to flash, and the trend data is sent to the database over the network.

9.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	CPU cost	Best suited to
Programmed I/O	The CPU repeatedly tests the device's status and copies each word itself (busy-wait).	Highest	Very simple, fast devices; boot code.
Interrupt-driven I/O	The device interrupts the CPU when a byte or word is ready; the CPU copies it.	One interrupt per word or byte	Low-rate, event-driven devices: bedside sensors, keyboards, network card signalling.
DMA	The CPU gives the DMA controller a memory address and a length; the controller moves the whole block between the device and memory and interrupts once at the end.	One interrupt per block	High-volume devices: MRI scanner, disks, network at high rate.

Choice: DMA for the MRI scanner, interrupt-driven I/O for the bedside sensors (low data rate but a tight deadline), and DMA for the database server's disks.

9.4 Effect of caching on record look-up speed

Effective access time EAT = h × t_cache + (1 − h) × t_slow. For a record look-up with t_cache = 100 ns (RAM), t_slow = 100 µs (SSD) and hit ratio h = 0.95:

```
EAT = 0.95 x 0.1 us + 0.05 x 100 us = 0.095 + 5.0 = about 5.1 us
Without cache (always SSD): 100 us   ->   about 20x slower
```

Even a few misses dominate, so raising the hit ratio is the best way to speed up look-up. Patient look-ups show locality (current in-patients are read repeatedly), so the hit ratio is high.

9.5 Alternative concept – interrupt-driven I/O (one interrupt per word) for the MRI scanner

Solution: the scanner interrupts the CPU for every 4-byte word, and the interrupt handler copies the word into memory. Compared with DMA, which transfers blocks of 64 KB with one interrupt per block, for a 500 MB image (500 × 1,048,576 = 524,288,000 bytes):

Criterion	Interrupt per word	DMA (64 KB blocks)
Number of interrupts	524,288,000 / 4 = 131,072,000 interrupts. O(n).	524,288,000 / 65,536 = 8,000 interrupts. O(n/block).
CPU time (assuming about 2 µs per interrupt, including context save/restore and the copy)	131,072,000 × 2 µs ≈ 262 s, over 4 minutes of CPU time just for moving data, which blocks other jobs.	8,000 × 2 µs = 16 ms for interrupts; the copying is done by the DMA controller, leaving the CPU free for reconstruction.
Memory buffers	The 500 MB image buffer in RAM, plus a word register in the device. The handler copies into the buffer.	The same 500 MB image buffer, plus a block buffer (or two for double buffering, 2 × 64 KB) that the DMA controller fills.

(The 2 µs per interrupt is a typical assumption.) **Conclusion:** at MRI data rates, interrupt-per-word I/O would consume the CPU, so DMA is the only practical method.

Task 10 – Measure and Compare (4 Marks)

All measurements were made by Task10/task10_bench.c, compiled with gcc 15.2.0 (-std=gnu11 -Wall -Wextra -pthread -O0) and run on Ubuntu 26.04.1 LTS (WSL2) with 20 virtual CPUs. Times were taken with clock_gettime (wall time) and getrusage (user and kernel time). Running the whole program under the time command gave real 5.268 s, user 1.551 s and sys 3.237 s, so about 68% of its CPU time was spent in kernel mode, which agrees with the split in the tables below.

10.1 System calls made by the copy program (buffer size)

```
gcc -std=c11 -Wall -Wextra -pthread task06_copy.c -o task06
strace -c ./task06        # summary table: calls per system call
strace -e trace=read,write ./task06 2>&1 | head   # individual calls
```

Buffer (bytes)	read() calls (program)	Expected ⌈n/b⌉ + 1	strace -c read count
1	10,001	10,001	10027 (total for all three runs, from strace -c)
512	21	21	(included in the total)
4,096	4	4	(included in the total)

10.2 Time in user mode and kernel mode

For the 4 MiB copy test with read()/write() (from Task 6):

Buffer (bytes)	Wall (s)	User (s)	Kernel (s)	Share of time in the kernel
1	3.145	0.592	2.521	about 80% of wall time; 81% of CPU time
64	0.097	0.012	0.047	about 80% of CPU time
4,096	0.009	0.000	0.004	—
65,536	0.005	0.000	0.003	—

With a 1-byte buffer the program spends most of its CPU time in kernel mode, which is the overhead of 8 million trap/return pairs (4.2 million reads and 4.2 million writes). With larger buffers the total time and the kernel share both fall sharply. (Wall time is about equal to user + kernel because the copy loop is CPU-bound and single-threaded.)

10.3 Time to create 1,000 processes and 1,000 threads

Operation (× 1,000)	Wall (s)	User (s)	Kernel (s)	Per unit
fork() + waitpid()	0.223	0.004	0.124	about 223 µs
pthread_create() + join	0.081	0.000	0.042	about 81 µs
posix_spawn("/bin/true") + wait	0.603	0.004	0.083	about 0.60 ms
fork() + exec("true") + wait	0.679	0.007	0.149	about 0.68 ms

10.4 How the measurements support or contradict my earlier judgements

Earlier judgement	Measurement	Verdict
System calls cost much more than function calls (Task 6).	About 107× (150 ns vs 1.4 ns).	Supported.
Large buffers cut system calls and time; use a 64 KB backup buffer (Task 6).	4,194,305 calls → 65 calls; 3.145 s → 0.005 s; no gain beyond 64 KB.	Supported.

Earlier judgement	Measurement	Verdict
mmap reduces calls so it should be faster (Task 6).	0.060 s vs 0.005–0.009 s for read/write with a large buffer.	Contradicted for a small 4 MiB file: page-fault cost outweighs the saved copy.
Library buffering approximates a large buffer (Task 5).	1-byte fread/fwrite 0.125 s vs 3.145 s for 1-byte read/write (25×).	Supported.
Threads are cheaper to create than processes (Task 7).	81 µs vs 223 µs, about 2.8×.	Supported.
posix_spawn is faster than fork+exec (Task 7).	0.603 s vs 0.679 s, only about 11%.	Mostly contradicted: the cost is loading the new program, not fork.

10.5 Alternative concept – threads instead of processes

Criterion	1,000 threads	1,000 processes
Creation time	0.081 s (about 81 µs each).	0.223 s (about 223 µs each), about 2.8× slower. Both O(k) for k units.
Memory used	Each thread needs only its own stack (and a small kernel structure). Code, heap, data and open files are shared.	Each process needs its own task structure and page tables, plus a private copy of every page it later modifies (copy-on-write).
What is shared	Address space, open files, signal handlers; separate stack, registers and thread ID.	Nothing by default; separate address space and file descriptor table (copied at fork).

Explanation of the difference: a thread is created by allocating a stack and a kernel thread structure; no memory-management structures are copied. fork() must duplicate the page tables and mark every page copy-on-write, and later page faults copy the pages that are written. The price of sharing is isolation: a bug in one thread can corrupt every other thread in the process.

Task 11 – Recommend and Defend the Operating-System Plan (4 Marks)

11.1 Recommended plan

Environment	OS type	Structure	Key services	Protection	I/O method
Bedside monitors	Hard real-time (RTOS)	Microkernel (QNX-style) or minimal FreeRTOS	Fixed tasks, sensor I/O, watchdog, small log, secure link to nurse station	Signed firmware, memory protection (MPU), no user login	Interrupt-driven
Nurse tablets	Handheld/mobile	Layered (Android-type)	Touch GUI, Wi-Fi, encrypted storage, power management	App sandbox, device encryption, remote wipe, MFA	Interrupt-driven
Doctor workstations	Desktop, multi-user	Monolithic with modules (Linux) or hybrid (Windows)	GUI, file system, printing, network	Login, ACLs, audit, disk encryption	Interrupt-driven + DMA
Database server	Clustered with fail-over	Modular monolithic (Linux)	Journalled file system, replication, accounting and audit	Strong authentication, encryption, ACLs, audit	DMA
Imaging server	SMP multiprocessor	Modular monolithic (Linux)	Many-core scheduling, large-file I/O, job queue	Per-user access, encrypted storage	DMA
Partner link	Distributed	Gateway on Linux	Secure messaging, queue and retry	TLS with mutual authentication, audit	Interrupt-driven + DMA

11.2 Evidence supporting the plan

- **Analysis (Tasks 1–4):** the ranked requirements show that only the monitor needs a hard deadline, so only the monitor needs an RTOS; a GPOS gives an unbounded (O(n)) delay.
- **Measurements (Tasks 6 and 10):** system calls cost about 107× an ordinary function call, which supports a small, lean call path on the monitor and large buffers (64 KB) on the servers.
- **Calculation (Task 2):** two nodes raise availability from 99.9% to about 99.9999%, which supports the cluster.
- **Calculation (Task 9):** DMA needs 8,000 interrupts for a 500 MB image against 131 million for interrupt-per-word.

11.3 Limitations and risks

- **Complexity:** four OS families (RTOS, mobile, desktop, server) need several skills and update processes.
- **Microkernel overhead:** message passing costs 2–10× per request on the monitors; acceptable because it is far below 10 ms, but it must be verified on the real hardware.
- **Cluster:** fail-over adds a few seconds and needs careful split-brain protection; replication doubles storage.
- **Measurement limits:** my measurements are from a one-CPU Linux machine and a 4 MiB file; the hospital's hardware, file sizes and loads differ and need re-testing.
- **Security:** every extra interface (partner link, tablets) enlarges the attack surface and needs patching.

11.4 Suitability for a group of three hospitals

The plan scales by replication: each hospital keeps its own monitors, tablets and workstations (they scale linearly with the number of devices), and the cluster grows by adding nodes. The partner link becomes a

distributed system with shared identity and replicated records. The weak points are consistency between sites and the wide-area network; these are tackled in Task 12.

11.5 Alternative concept – virtual machines on two large servers

Solution: each environment's software (workstation images, database, imaging, partner gateway) runs in virtual machines on two large physical servers under a hypervisor. Bedside monitors and tablets remain physical devices.

Criterion	Virtual machines on two servers	Separate physical machines
Response time for the monitors	Any monitor logic moved into a VM is subject to hypervisor scheduling, which can pause a virtual CPU for several milliseconds up to a time slice, plus the network. A 10 ms guarantee is no longer possible, so the monitors' control must stay on the device.	The monitor's RTOS runs directly on hardware; its latency is bounded.
Memory and CPU overhead	Typically 2–10% CPU for CPU-bound work and more for I/O; every VM needs its own guest OS memory (about 0.5–2 GB), so memory grows as O(k × M_guest) for k VMs.	No virtualisation overhead; but hardware sits underused.
Reliability	Management is easier (snapshots, live migration), but a host failure takes down every VM on it, so all VMs must be restarted on the other server; each server must be sized to carry the full load.	Faults are contained to one machine; more machines to maintain.

Conclusion: virtualise workstation images and the gateway to simplify administration, but keep the monitors' real-time control on dedicated hardware and place the database cluster nodes on separate hosts so that one host cannot take down both.

Task 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: remote monitoring devices in patients' homes send readings over the internet, and patient records move to a cloud service shared with the partner hospitals.

A. Impact Analysis (2 Marks)

Part of the design	Impact	How it is affected
Bedside monitors (RTOS)	None	Still a hard real-time, on-site system on the hospital network.
Home monitoring device (new)	New environment	Battery-powered, always connected over an untrusted network, installed in homes where no IT staff are present. Needs an embedded RTOS, secure remote updates and local alarming.
Tablets and workstations	Modified	They now reach records through the internet, so they need TLS, multi-factor authentication and offline caching in case the connection drops.
Database cluster	Modified	No longer the only copy of the records; it becomes one node of a distributed record service, so consistency and replication enter the design.
Partner link	Redesigned	A point-to-point link is replaced by a shared cloud service with identities from three organisations.
Imaging server	Retained	Still a local SMP machine; large images stay on-site and the cloud holds only references and reports (bandwidth and cost).
Protection mechanisms	Strengthened	Device authentication (certificates), secure boot and signed firmware, encryption in transit and at rest, per-organisation access control and cross-organisation audit logs.
Services	Changed	Communication, error detection (retries, store-and-forward), security and accounting become more important; user interface and file services on the home device remain minimal.

B. Evaluation of the Original Design (2 Marks)

Criterion	Can the original design support it?	Evidence
Response time	No for the home alarm. The 10 ms guarantee was achieved by a dedicated RTOS on a local bus; internet latency is variable (tens to hundreds of ms) and cannot be bounded. The original design had no remote real-time path.	Task 1 ranked a guaranteed deadline first. Task 11 showed that adding a hypervisor or network between sensor and alarm breaks the guarantee.
Reliability	Partly. The on-site cluster is highly available (99.9999%), but home links fail more often, and a cloud dependence adds a new failure source: the hospital must keep working when the internet is down.	Task 2 availability calculation; single site, single cluster.
Security	No. The original trust boundary was the hospital network plus one partner link. Thousands of internet-reachable home devices and a multi-tenant cloud greatly increase the attack surface.	Task 4 protection mechanisms covered users and processes, not device identity or cross-organisation access.

Verdict: the original design cannot be simply retained, but its core remains sound, so it should be modified rather than redesigned from scratch.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY. Keep the proven hospital core, add two new layers, and redesign only the partner link.

Component	Decision	Revised design
Bedside monitors	RETAIN	Microkernel/embedded RTOS unchanged.
Home device	NEW	Embedded RTOS (microkernel or FreeRTOS-class). The alarm threshold is evaluated on the device itself within the 10 ms budget, so the deadline does not depend on the internet. Interrupt-driven sensor I/O and deep sleep give long battery life. A local buffer stores readings when offline and sends them later (store-and-forward). Secure boot, device certificate, TLS and signed over-the-air updates.
Ingestion gateway	NEW	A scalable server cluster (Linux) receives readings over TLS and forwards alerts to clinicians within seconds, as a soft real-time service.
Records service	MODIFY (distributed)	Records are replicated across availability zones in the cloud, with strong consistency for the clinical record. The hospital's database cluster remains as a local replica so the hospital works during an internet outage.
Partner sharing	REDESIGN	Replaced by role-based access in the shared cloud service with identity federation, encryption and audit logging; each hospital sees only authorised patients.
Tablets and workstations	MODIFY	Add TLS, multi-factor authentication and local encrypted cache.
Imaging server	RETAIN	Unchanged; reports and image references are shared via the cloud.

D. Evidence-Based Defence (2 Marks)

- **Why retain the core:** the analysis in Tasks 1–4 and the calculations in Tasks 2 and 9 do not depend on the changed requirement; monitors, imaging and workstations are unaffected, so redesigning them would add cost and risk with no benefit.
- **Why the home alarm is local:** the ranked requirements (Task 1) and the polling/interrupt analysis (Task 8) show that a bounded response needs an RTOS and interrupts at the point of measurement. A cloud round trip cannot give a guarantee, so the cloud is used for recording and escalation, not for the first alarm.
- **Why a hybrid records design:** it keeps the hospital working if the internet fails (reliability) while sharing data through the cloud, at the price of replication traffic and consistency work.
- **Why not redesign everything:** moving all processing to the cloud would remove the on-site fail-over and put the 10 ms alarm behind the internet, which the measurements and calculations show is not safe.

Option	Safety	Cost and effort	Risk
Retain unchanged	Home devices unsupported; no remote alarm	Lowest	Fails the new requirement
Modify (chosen)	Local alarm, on-site resilience, cloud sharing	Moderate: new device OS, gateway, cloud, security work	Manageable: new attack surface, replication consistency
Redesign completely	Alarm depends on the network	Highest	High: unproven, higher outage risk

Trade-offs accepted: higher operating cost, more security work (certificates, patching, audit), replication and consistency complexity, and a dependence on privacy law compliance for shared cloud data. **Final decision:** modify the original plan as described in C.

Appendix – GitHub Repository and How to Reproduce

Repository: https://github.com/mdfalahmiya-lang/OS/tree/main (shared with the course faculty). One folder per task, at least one commit per task.

```
OS/ (repository root)
  README.md              name, register no., gcc 15.2.0, Ubuntu 26.04.1, commands, results
  Task01/ ... Task12/    one folder per task (notes, source, outputs, screenshots)
  Task05/task05_demo.c   strace demo (raw system calls and fopen/fread)
  Task05/syscall_cost.c  system call versus function call timing
  Task06/task06_copy.c   completed copy program (TODO 1)
  Task07/task07_shell.c  completed shell program (TODO 1)
  Task07/task07_dup.c    experiment: second fflush removed, exit(3)
  Task10/task10_bench.c  buffer-size, process/thread and mmap measurements
```

Commands

```
gcc -std=c11 -Wall -Wextra -pthread task06_copy.c -o task06 && ./task06
gcc -std=c11 -Wall -Wextra task07_shell.c -o task07 && ./task07
gcc -std=c11 -Wall -Wextra task07_dup.c -o task07_dup && ./task07_dup > out.txt
gcc -std=gnu11 -Wall -Wextra -pthread task10_bench.c -o task10 && ./task10
strace -c ./task06      # system-call counts
time ./task10           # real, user and sys time
```